# Random Variables

Math 5- Dr- Lamia
AlRefaai

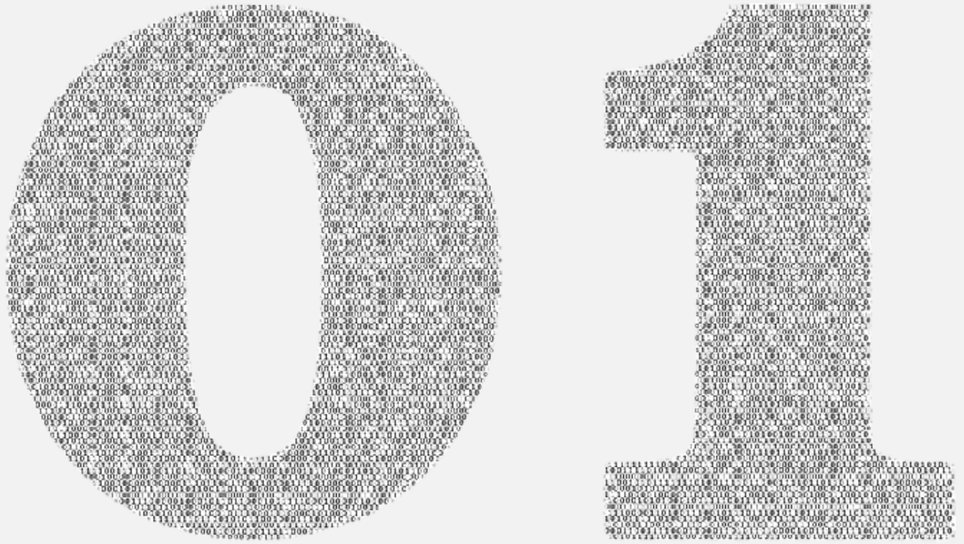
# Contents



*-In the realm of practicality, each participant plays a crucial role, contributing substantively to the real-life applications at hand*

# Bernoulli’s Distribution

The Bernoulli distribution is relevant to situations involving a single trial with two potential outcomes, commonly referred to as Bernoulli trials. Imagine any experiment posing a binary question such as whether a coin will land on heads, if a die will roll a six, if an ace will be picked from a deck of cards, or if voter X will vote "yes" in a political referendum, among others.



# Slight Elucidation

Given its binary nature, denoted by the variable X representing a random variable conforming to the Bernoulli's distribution, the probability mass function of X elegantly manifests as either 0 or 1.

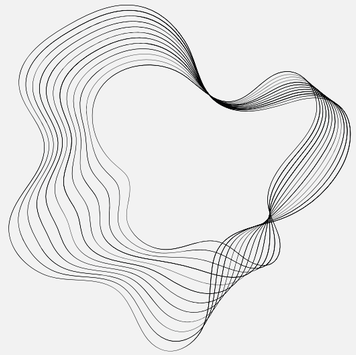
$*$ LET $X = 1$ if a success occurs, and $X = 0$ if a failure occurs.

$$P(X = x) = p^x(1-p)^{1-x}$$

$$for\ x = 1, P(X = 1) = p^1(1-p)^0 = p$$

$$for\ x = 0, P(X = 0) = p^0(1-p)^1 = 1-p$$

that's totally comprehensible, since both are complements of each other.

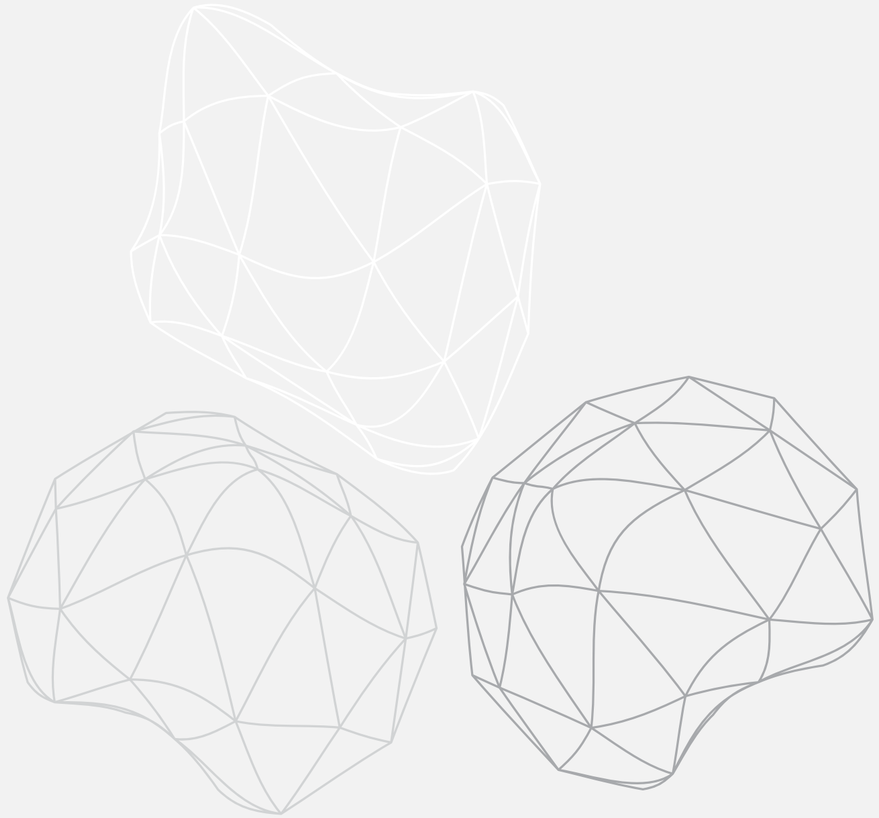


# Real life application

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import bernoulli

# Set the probability of success for the treatment
p_success = 0.8  # You can adjust this probability based on your scenario

# Number of patients
num_patients = 1000

# Generate synthetic data using Bernoulli distribution
treatment_outcomes = bernoulli.rvs(p_success, size=num_patients)

# Calculate success and failure counts
success_count = np.sum(treatment_outcomes == 1)
failure_count = np.sum(treatment_outcomes == 0)

# Calculate expectation (mean) and variance
expectation = p_success
variance = p_success * (1 - p_success)

# Display the results
print(f"Success Count: {success_count}")
print(f"Failure Count: {failure_count}")
print(f"Expectation (Mean): {expectation}")
print(f"Variance: {variance}")

# Plot the distribution
plt.bar(['Success', 'Failure'], [success_count, failure_count])
plt.title('Distribution of Treatment Outcomes')
plt.xlabel('Outcome')
plt.ylabel('Count')
plt.show()
```

# Geometric Distribution

The geometric distribution is a discrete probability distribution focused on the likelihood of the first success within a specific trial. It addresses questions such as the probability of achieving the first success in various scenarios, such as rolling a six in a series of die rolls, contracting the flu in a given year, gaining support for a law in repeated interviews, encountering a defective product in a random sample, or achieving success in a project or task. This distribution is applicable when determining the number of attempts required to achieve the initial successful outcome.



# Geometric Distribution

- The geometric distribution models the probabilities for the first success occurring on the $X^{th}$ trial. However, your data must meet the following requirements for the geometric distribution to be appropriate.
- **Your data must be binary**: For example, infected or uninfected, 6 or not 6, pass or fail, etc.
- **Independent trials:** One trial's result does not affect the next trial. For example, a coin toss doesn't affect the following coin toss.
- **The probability remains constant over time**. In some contexts, this supposition is true due to the physical attributes of the process, such as coin tosses and die rolls. However, the likelihood won't necessarily remain constant in other contexts. For example, the probability of a product defect at a manufacturing plant can change over time. Use a P chart (a control chart) to verify this assumption when the chance can change.

# Geometric Distribution-Formula

- The geometric distribution formula for the probability of the first success occurring on the $X^{th}$ trial is the following:

- The mean of a geometric distribution is 1 / P. This mean is the expected value for a geometric distribution

$$P(x) = (1 - p)^{x-1}p$$

$$\mathbb{E}[X^2] = \frac{2}{p^2} - \frac{1}{p},$$

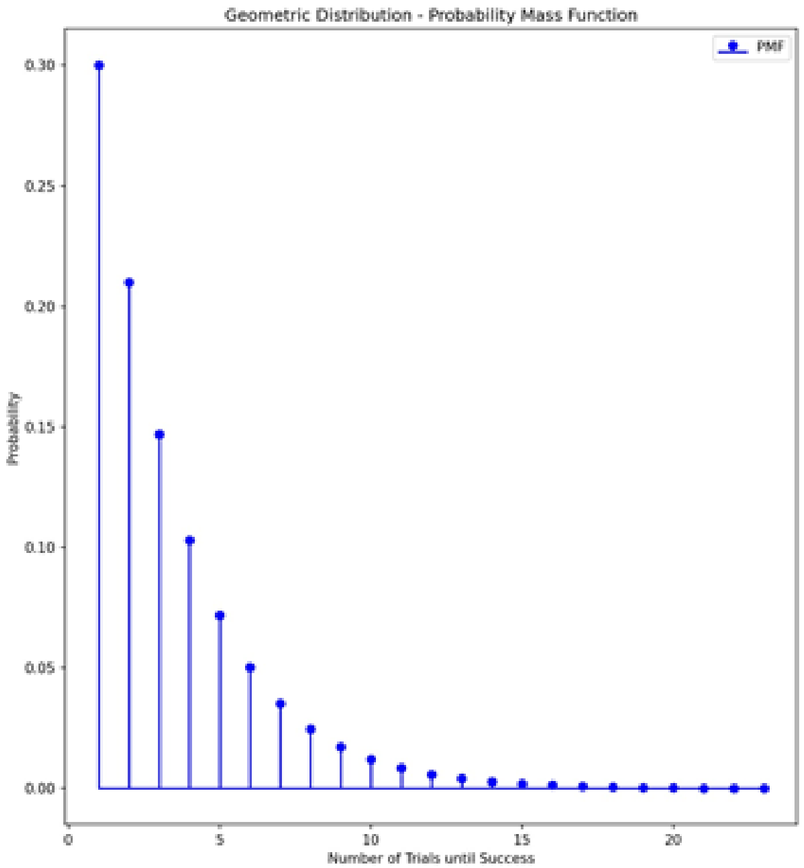
$$\operatorname{Var}[X] = \frac{1-p}{p^2}.$$

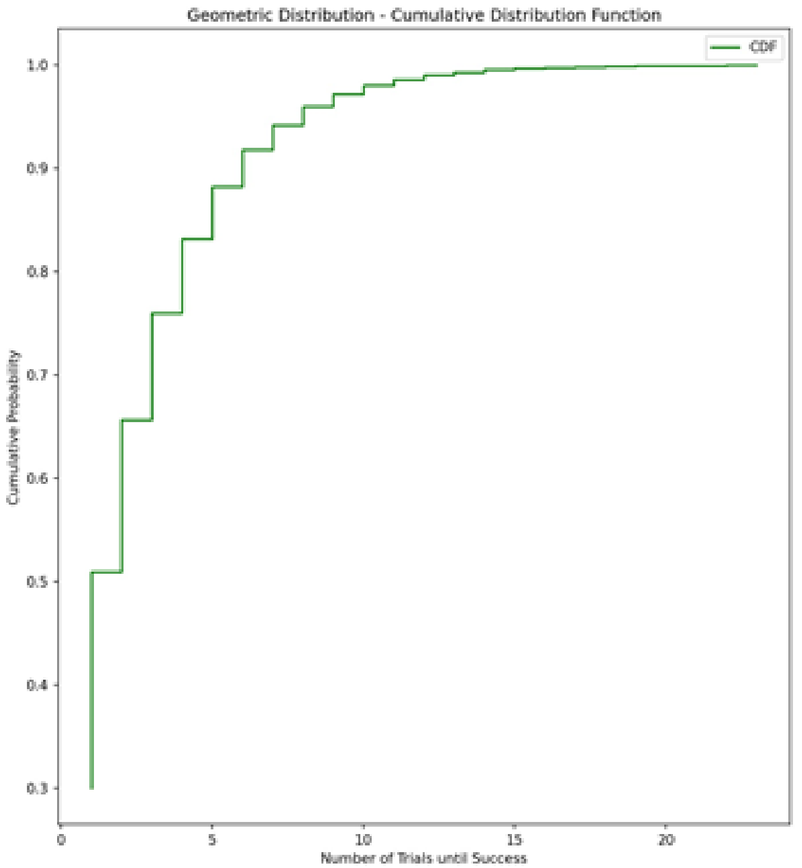
# Code implementation:-

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import geom

def generate_geometric_random_variables(prob_success, size=1):

    # Checking the probability range
    if prob_success <= 0 or prob_success > 1:
        raise ValueError("Probability of success must be in the range (0, 1].")

    # Generating geometric random variables
    geometric_values = np.random.geometric(p=prob_success, size=size)

    return geometric_values

def geometric_pmf(p, k):

    pmf = {(1 + i): p * (1 - p)**i for i in range(k)}
    return pmf

def geometric_cdf(p, k):

    pmf = geometric_pmf(p, k)
    cdf = {key: sum(pmf[i] for i in range(1, key+1)) for key in pmf}
    return cdf

def calculate_statistics(pmf):

    e_x = sum(value * probability for value, probability in pmf.items())
    e_x_squared = sum(value**2 * probability for value, probability in pmf.items())
    var_x = e_x_squared - e_x**2
    return e_x, e_x_squared, var_x

def plot_geometric_distribution(prob_success, size=1000):
    # Generate random variables
    random_variables = generate_geometric_random_variables(prob_success, size)
    # Calculate the probability mass function (PMF) and cumulative distribution function (CDF)
    x = np.arange(1, np.max(random_variables) + 1)
    pmf = geom.pmf(x, p=prob_success)
    cdf = geom.cdf(x, p=prob_success)
# Plot PMF on the left subplot
    plt.figure(figsize=(12, 6))
    plt.subplot(1, 2, 1)
    plt.stem(x, pmf, basefmt="b", linefmt="b-", markerfmt="bo", label='PMF')
    plt.title('Geometric Distribution - Probability Mass Function')
    plt.xlabel('Number of Trials until Success')
    plt.ylabel('Probability')
    plt.legend()
    # Plot CDF on the right subplot
    plt.subplot(1, 2, 2)
    plt.step(x, cdf, color='green', label='CDF')
    plt.title('Geometric Distribution - Cumulative Distribution Function')
    plt.xlabel('Number of Trials until Success')
    plt.ylabel('Cumulative Probability')
    plt.legend()
    plt.tight_layout()
    plt.show()
# Input probability of success
prob_success_input = float(input("Enter the probability of success (0 < prob_success <= 1): "))
# Input number of random variables to generate
num_variables_input = int(input("Enter the number of random variables to generate: "))
# Set the number of trials (you can adjust this based on your needs)
k_trials = 10
p_success = prob_success_input
pmf = geometric_pmf(p_success, k_trials)
cdf = geometric_cdf(p_success, k_trials)
# Calculate statistics
expected_value, expected_value_squared, variance = calculate_statistics(pmf)
# Print statistics
print("Probability Mass Function (PMF):", pmf)
print("Cumulative Distribution Function (CDF):", cdf)
print("Expected Value (E(X)): {:.2f}".format(expected_value))
print("Expected Value of X^2 (E(X^2)): {:.2f}".format(expected_value_squared))
print("Variance (Var(X)): {:.2f}".format(variance))
```

```python
print("Variance (Var(X)): {:.2f}".format(variance))
# Plot the Geometric Distribution (PMF and CDF)
plot_geometric_distribution(prob_success_input, size=num_variables_input)
```

# Output:-




```
Enter the probability of success (0 < prob_success <= 1): 0.3
Enter the number of random variables to generate: 1000
Probability Mass Function (PMF): {1: 0.3, 2: 0.21, 3: 0.14699999999999996, 4: 0.10289999999999998, 5: 0.07202999999999998, 6: 0.05042099999999998, 7: 0.035294699999999984, 8: 0.024706289999999999, 9:
Cumulative Distribution Function (CDF): {1: 0.3, 2: 0.51, 3: 0.657, 4: 0.7599, 5: 0.8319300000000001, 6: 0.882351, 7: 0.9176457, 8: 0.94235199, 9: 0.959646393, 10: 0.9717524751}
Expected Value (E(X)): 2.96
Expected Value of X^2 (E(X^2)): 13.65
Variance (Var(X)): 4.91
```

# Binomial Distribution

A distribution quantifies the number of trials or observations, each with an equal probability of a particular outcome. The Binomial distribution calculates the probability of observing a specified number of successful outcomes in a set number of trials. Key parameters are N, P, and Q, with equations illustrated. Real-life applications include assessing raw and used materials in manufacturing and conducting surveys to gauge positive and negative public reviews for a product or place. All of the illustrations are provided in the following slide.

BINOMIAL

## Binomial Distribution Formula

$$P(X) = {}_nC_x\, p^x (1-p)^{n-x}$$

# BINOMIAL

Mean = np

Variance = npq

# Binomial's Code Implementation

```python
import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import binom

# Set the number of trials and probability of success
n = 10
p = 0.3

# Generate the random variable
rv = binom(n, p)
print(f'Number of trials: {n}')
print(f'Random variable value: {rv.rvs()}')

# Plot the PMF
x = np.arange(rv.ppf(0.01), rv.ppf(0.99))
plt.plot(x, rv.pmf(x), 'bo', ms=8, label='binom pmf')
plt.vlines(x, 0, rv.pmf(x), colors='b', lw=5, alpha=0.5)
plt.xlabel('Number of Successes')
plt.ylabel('Probability')
plt.title('Binomial PMF')
plt.show()

# Plot the CDF
x = np.arange(rv.ppf(0.01), rv.ppf(0.99))
plt.plot(x, rv.cdf(x), 'bo', ms=8, label='binom cdf')
plt.vlines(x, 0, rv.cdf(x), colors='b', lw=5, alpha=0.5)
plt.xlabel('Number of Successes')
plt.ylabel('Probability')
plt.title('Binomial CDF')
plt.show()

# Compute the expectation and variance
print(f'Expectation: {rv.mean()}')
print(f'Variance: {rv.var()}')
```

# Poisson Distribution

A Poisson distribution is a discrete probability distribution. It gives the probability of an event happening a certain number of times within a given interval of time or space and it has one parameter λ (lambda).

POISSON DISTRIBUTION

$$px(k) = \frac{\lambda^k}{k!} e^{-\lambda} \qquad \text{Where} \quad k = 1, 2, \ldots\ldots$$

$$Fx(k) = P[X \leq k] = \sum_{l=0}^{k} \frac{\lambda^l}{l!} e^{-\lambda}$$

# Poisson Distribution

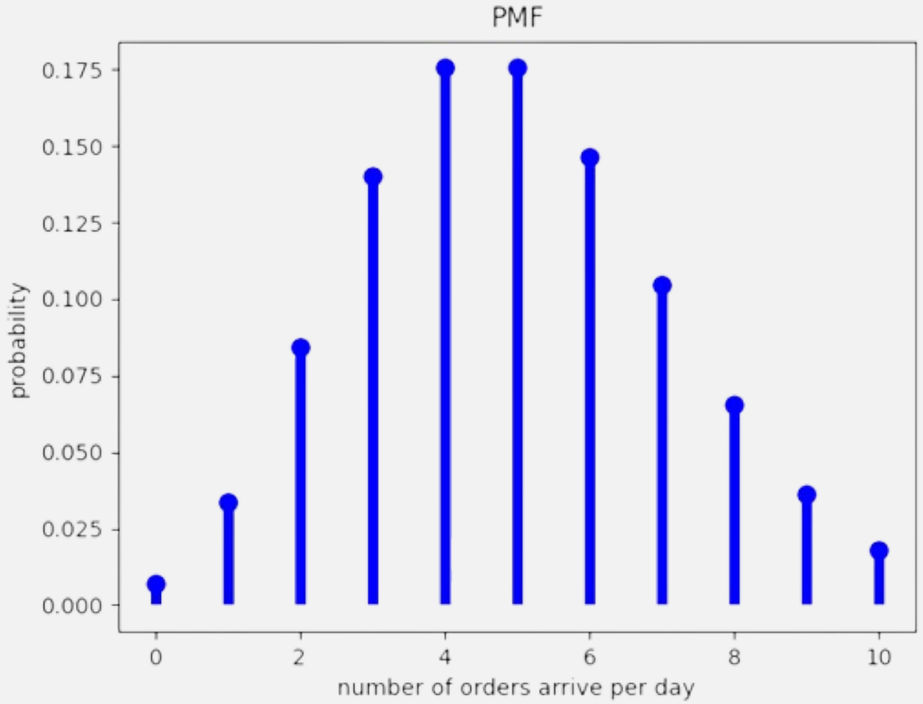
## Presenting questions that yield valuable insights:

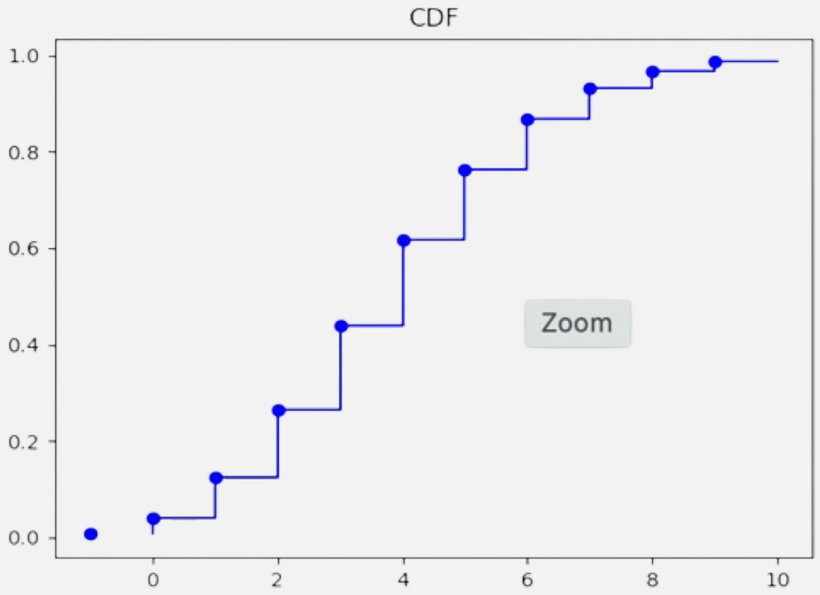
-Under what circumstances is the Poisson distribution utilized?
-How is the Poisson distribution utilized?
-What are some real-life applications in which we can employ the Poisson distribution?


Of course, we will integrate this with a code implementation and visual representations to enhance our comprehension further.

# Implementation:

```python
# poisson application
import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats
lambd=5
x = np.arange(0,11)
rv = stats.poisson(lambd)
f = rv.pmf(x)
r = np.cumsum(f)
plt.step(x,r,color='b')
plt.step(x-1,r, *args: 'bo', lw=5,color='b')
plt.title("CDF")
plt.show()
plt.plot( *args: x, f, 'bo', ms=8,color='b')
plt.vlines(x, ymin: 0,f,color='b', lw=5)
plt.xlabel("number of orders arrive per day")
plt.ylabel("probability")
plt.title("PMF")
plt.show()
```

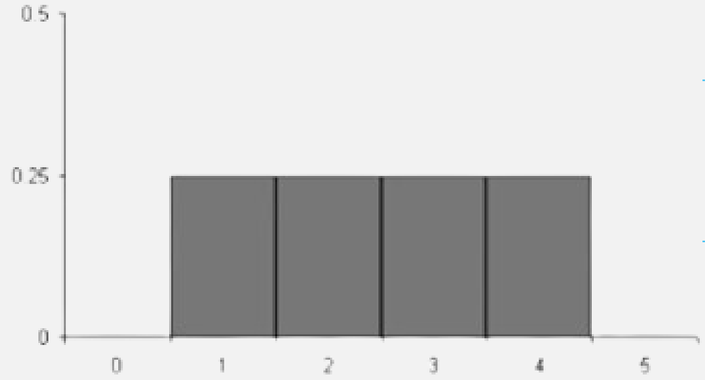


# Uniform Distribution-Discrete

The discrete uniform distribution assigns equal probabilities to n events.. The corresponding continuous probability distribution is the (continuous) uniform distribution

$$P(X = x) = \begin{cases} \frac{1}{n}, & \text{if } x = x_1, x_2, \ldots, x_n \\ 0, & \text{if not} \end{cases}.$$

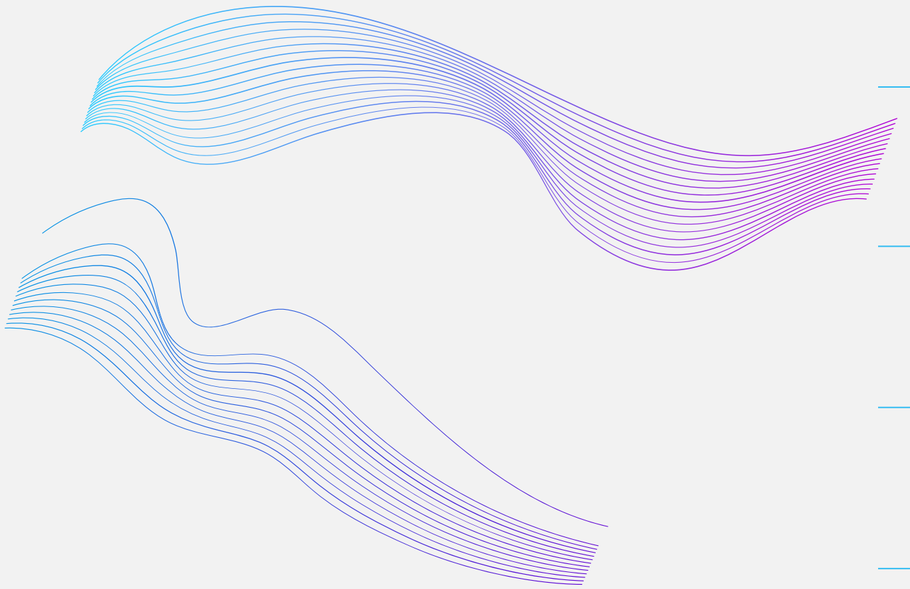


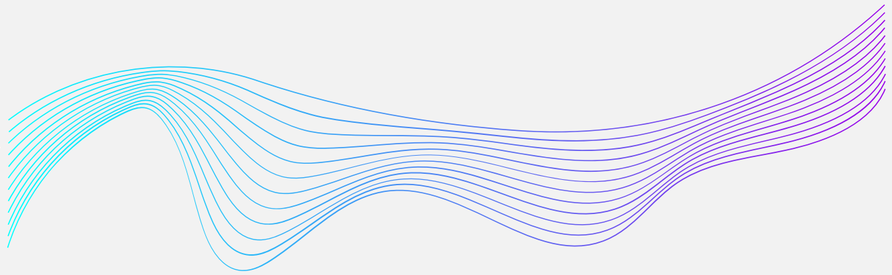

Discrete uniform distribution, $n = 4$

# Uniform Distribution-Discrete

## Posing inquiries from which we can derive meaningful insights:

-What are some of its properties?
-How can we benefit from Discrete Uniform Distribution?
-Where can we see it in real life?
-Ultimately, we can seamlessly integrate this concept into a tangible code implementation that will be presented in the following slide.





```python
# Set the range of the random variable
a = 1
b = 13

# Generate the random variable
X = np.random.randint(a, b+1, size=10000)

# Plot the PMF
counts, bins, _ = plt.hist(X, bins=np.arange(a-0.75, b+1.5,0.5), density=True)

plt.xlabel("x")
plt.ylabel("PMF")
plt.title("Probability Mass Function of Uniform Discrete Random Variable")
plt.show()

# Plot the CDF
counts, bins, _ = plt.hist(X, bins=np.arange(a-0.5, b+1.5), density=True, cumulative=True)
plt.plot(bins[:-1], counts, "o-", color='red')
plt.xlabel("x")
plt.ylabel("CDF")
plt.title("Cumulative Distribution Function of Uniform Discrete Random Variable")
plt.show()

# Compute the expectation and variance
expectation = (a + b) / 2
variance = ((b - a + 1) ** 2 - 1) / 12
print("Expectation:", expectation)
print("Variance:", variance)
```

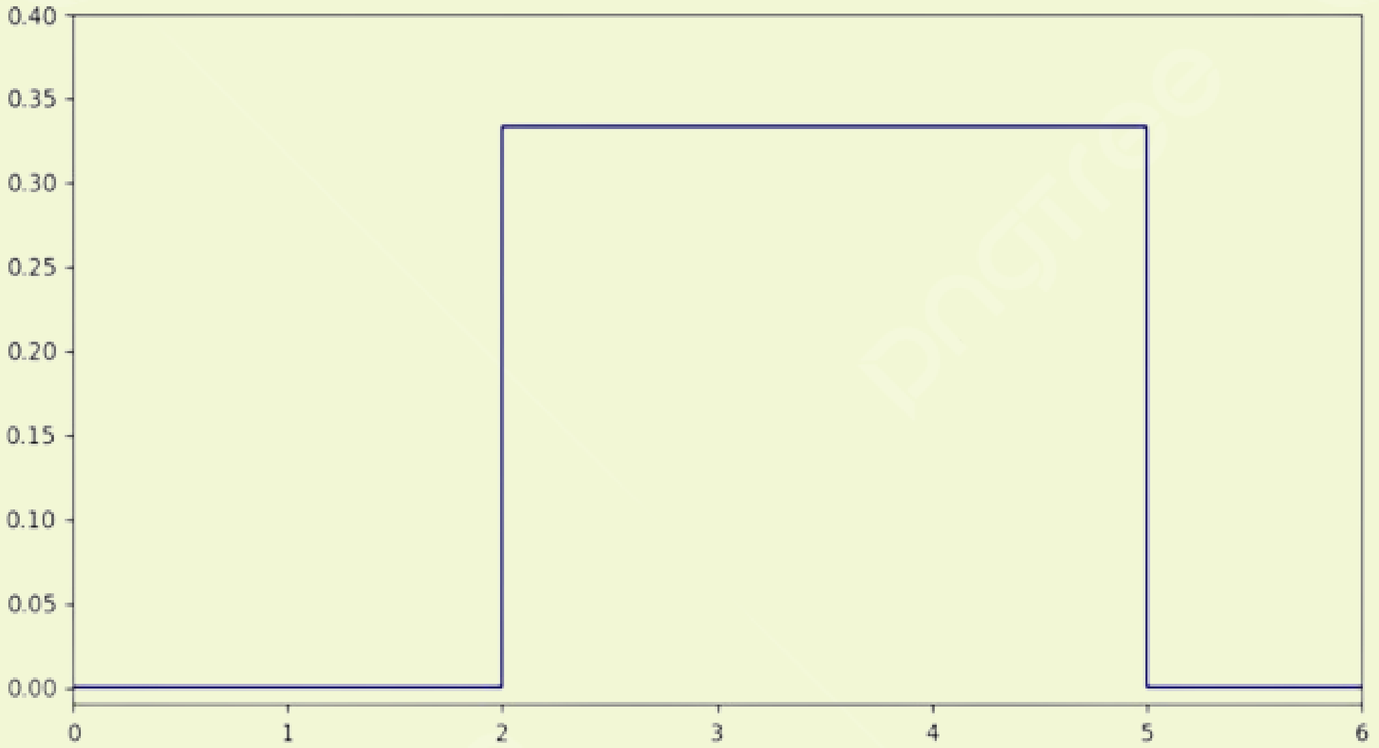
# Continous random variables

# Uniform-Continuous

# Introduction: -

The continuous uniform distribution, or rectangle distribution, is a symmetric family of probability distributions used to model experiments with arbitrary outcomes within specific bounds defined by parameters a and b. Represented as U(a,b), it can have open (]a,b[) or closed ([a,b]) intervals, and all intervals of the same length are equally likely on its support. This distribution, characterized by maximum entropy for a random variable X within its constraints, is a fundamental concept in probability theory and statistics.

## The following graph shows the distribution with a = 2 and b = 5:



# Definitions: -

- **Probability density function:**

$$f(x) = \begin{cases} \frac{1}{b-a} & \text{for } a \le x \le b, \\ 0 & \text{for } x < a \text{ or } x > b. \end{cases}$$

- **Cumulative distribution function:**

$$\begin{aligned} F_X(x) &= \int_{-\infty}^{x} f_X(t)\,dt \\ &= \int_{a}^{x} \frac{1}{b-a}\,dt \\ &= \frac{x-a}{b-a}, \qquad a \le x \le b. \end{aligned}$$

# Variables: -

- **The expected value:**

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x f_X(x)\,dx = \int_a^b \frac{x}{b-a}\,dx = \frac{a+b}{2},$$

- **The second moment of a uniform random variable is:**
-

$$\mathbb{E}[X^2] = \int_{-\infty}^{\infty} x^2 f_X(x)\,dx = \int_a^b \frac{x^2}{b-a}\,dx = \frac{a^2+ab+b^2}{3},$$

- **The variance of a uniform random variable is:**

$$\operatorname{Var}[X] = \mathbb{E}[X^2] - \mathbb{E}[X]^2 = \frac{(b-a)^2}{12}.$$

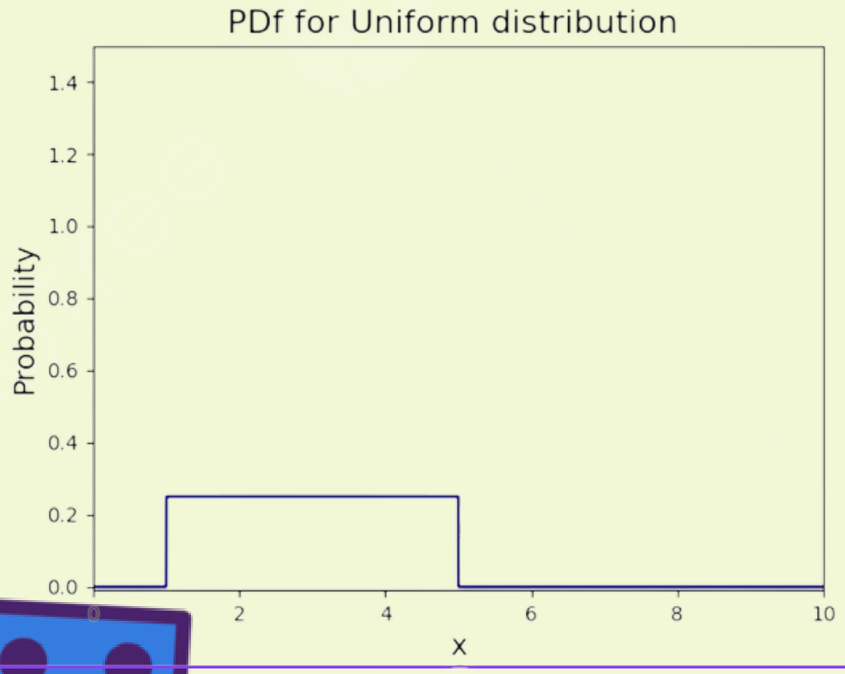
# Applications: -

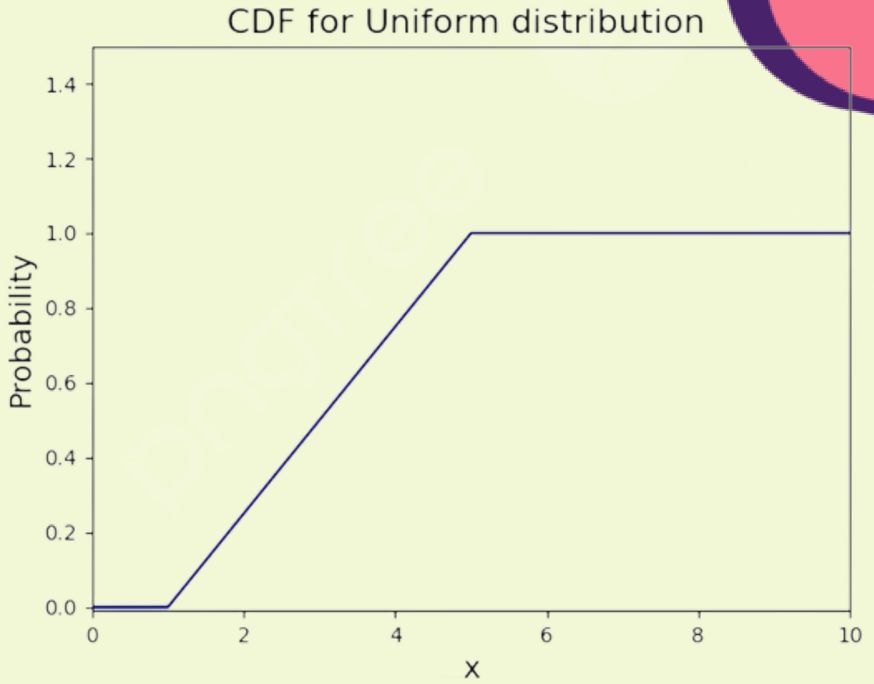
- Randomized Controlled Trials
- Simulations and Gaming
- Quality Control
- Random Number Generation
- Load balancing
- Lottery and Gambling
- Sports Scheduling
- Queuing Theory

# Code Implementation:-

```python
import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats

a = float(input("Enter minimum value: "))
b = float(input("Enter maximum value: "))
rand_unif = stats.uniform.rvs(loc=a, scale=(b-a), size=1000)

#PDf  for Uniform
plt.ylim(-0.01, 1.5)
plt.xlim(0, b+5)
plt.title("PDf for Uniform distribution ", fontsize =16)
plt.ylabel("Probability", fontsize =14)
plt.xlabel("x", fontsize =14)
x = np.arange(0, b+5, 0.001)
plt.plot(x, stats.uniform.pdf(x, loc=a, scale=(b-a)), color="darkblue")
plt.show()

#CDf  for Uniform
plt.ylim(-0.01, 1.5)
plt.xlim(0, b+5)
plt.title("CDF for Uniform distribution ", fontsize =16)
plt.plot(x, stats.uniform.cdf(x, loc=a, scale=(b-a)), color="darkblue")
plt.xlabel("x", fontsize =14)
plt.ylabel("Probability", fontsize =14)
plt.show()

M = np.mean(rand_unif )
V = np.var(rand_unif )
print("Mean= ",M)
print("variance= ",V)
```

# Output When a=1 and b=5:-




```
Enter minimum value: 1
Enter maximum value: 5
Mean=  3.034090198452379
variance=  1.291170790299683
```

# Exponential Distribution

## Why was the exponential distribution necessary to create?

- To estimate how long it will take for the next event—a success, failure, arrival, etc.—to occur.
- For instance, the following is what we hope to predict:
  The duration of time it takes a customer to complete their browsing and make a purchase from your store (success).

# Definitions:-

- **PDF:-**

Let $X$ be an exponential random variable. The PDF of $X$ is

$$f_X(x) = \begin{cases} \lambda e^{-\lambda x}, & x \geq 0, \\ 0, & \text{otherwise,} \end{cases}$$

where $\lambda > 0$ is a parameter. We write

$$X \sim \text{Exponential}(\lambda)$$

to say that $X$ is drawn from an exponential distribution of parameter $\lambda$

# Definitions:-

- **CDF:-**

The CDF of an exponential random variables can be determined by

$$F_X(x) = \int_{-\infty}^{x} f_X(t)dt$$
$$= \int_{0}^{x} \lambda e^{-\lambda t} dt = 1 - e^{-\lambda x}, \qquad x \geq 0.$$

Therefore, if we consider the entire real line, then the CDF is

$$F_X(x) = \begin{cases} 0, & x < 0, \\ 1 - e^{-\lambda x}, & x \geq 0. \end{cases}$$

# Applications:-

- A waiting time has an exponential distribution if the probability that the event occurs during a certain time interval is proportional to the length of that time interval.

- More precisely,  has an exponential distribution if the conditional probability $P(t<X<=t+\Delta t|X>t)$.

- **Memoryless property:-**
  One of the most important properties of the exponential distribution is the memoryless property:

  $P(X<= x+y \mid X>x) = P(X<=y)$

# Real life Example:-

- ***Time between calls (applied in the code)***

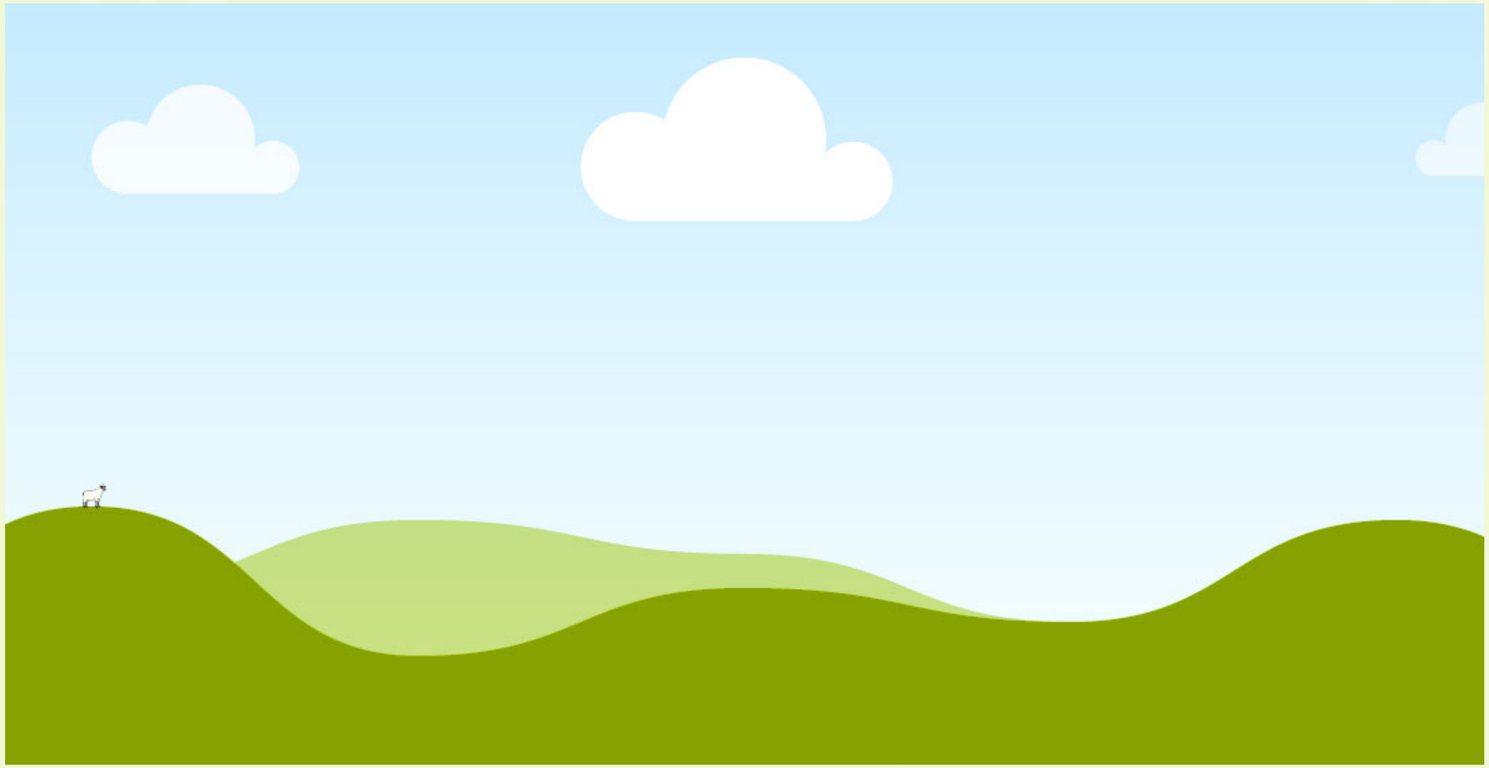
# Code Deployment:-

```python
import matplotlib.pyplot as plt
import numpy as np

# Rate parameter for the exponential random variable
lambda_ = 0.1

# Number of random variables to generate
n = 1000

# Generate n exponential random variables

x = np.random.exponential(1 / lambda_, n)
#np.random.exponential(scale, size): This function generates random numbers from an exponential distribution

x = x[(x > 10) & (x <= 15)]  # Filter values between 10 and 15
print(x)

# Compute the probability density function (PDF)
x_min, x_max = min(x), max(x)
bins = np.linspace(x_min, x_max, 50) #This line creates 50 evenly spaced bins & (np.linspace) is used to generate these bins.

pdf, _ = np.histogram(x, bins=bins, density=True)
#This line calculates the histogram of the data &&density=True parameter means that the area under the histogram will sum up to 1,


# Compute the cumulative distribution function (CDF)
cdf = np.cumsum(pdf * np.diff(bins))
#you're essentially summing up the areas of each histogram bin (calculated from the PDF)
#and accumulating these values to generate the cumulative distribution function (CDF)
```

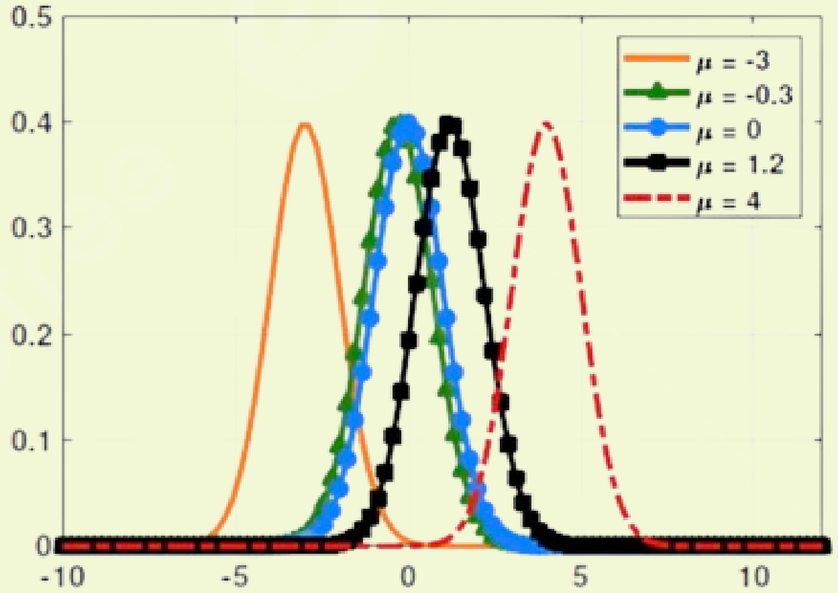
```python
# Compute the expectation of the exponential random variable
expectation = 1 / lambda_

# Compute the variance of the exponential random variable
variance = (1 / lambda_) **2

# Plot the PDF using fill_between
plt.fill_between(bins[:-1], pdf, 0)
plt.title('Probability Density Function (PDF)')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.show()

# Plot the CDF
plt.plot(bins[:-1], cdf, 'o-')
plt.title('Cumulative Distribution Function (CDF)')
plt.xlabel('x')
plt.ylabel('F(x)')
plt.show()

# Print the results
print(f'Expectation: {expectation:.2f}')
print(f'Variance: {variance:.2f}')
```

# Gaussian Distribution

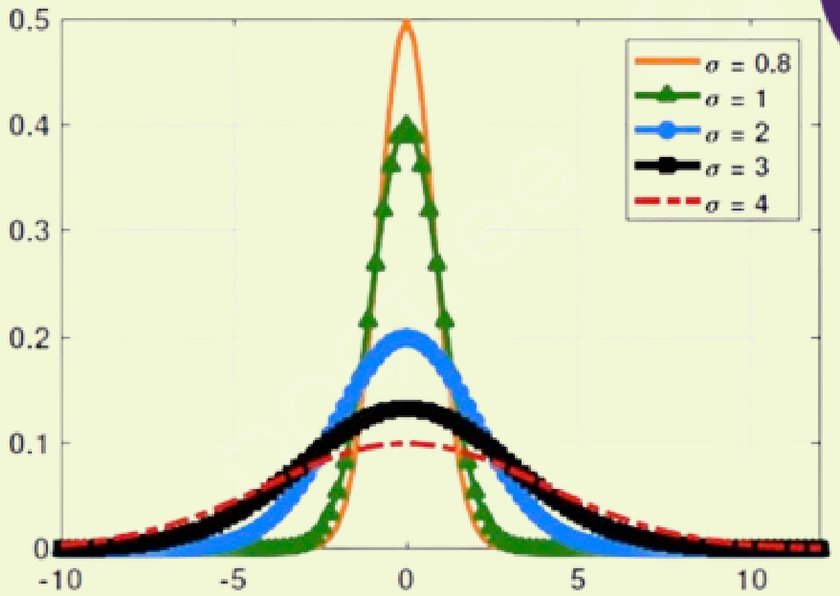
# ***What is Gaussian distribution?***



# As for the Graph representation of the PDF:-

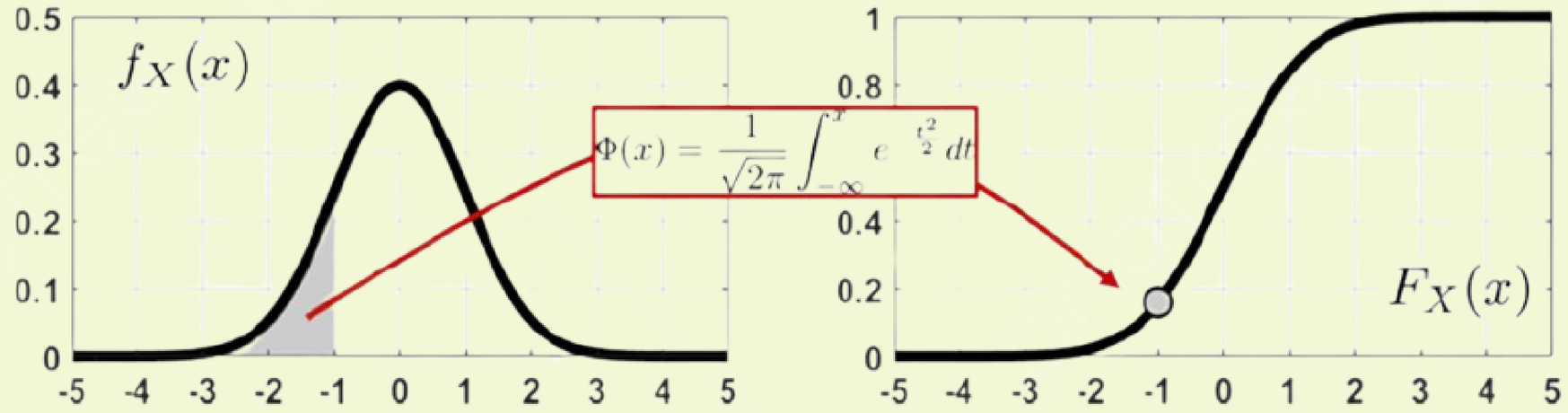



$\mu$ changes, $\sigma = 1$




$\mu = 0$, $\sigma$ changes

# The CDF of the Gaussian distribution:



# The Code implementation:-

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# Set the mean and standard deviation of the Gaussian distribution
mean = 0
std = 1

# Generate random samples from the Gaussian distribution
samples = np.random.normal(mean, std, size=10000)

# Plot the PMF of the samples
plt.hist(samples, bins=50, density=True, alpha=0.5)
plt.xlabel('Value')
plt.ylabel('Probability')
plt.title('PMF of Gaussian Continuous Random Variable')
plt.show()

# Plot the PDF of the samples
x = np.linspace(-5, 5, 100)
plt.plot(x, norm.pdf(x, mean, std))
plt.xlabel('Value')
plt.ylabel('Probability')
plt.title('PDF of Gaussian Continuous Random Variable')
plt.show()
```

```python
# Plot the CDF of the samples
plt.hist(samples, bins=50, density=True, alpha=0.5, cumulative=True
plt.xlabel('Value')
plt.ylabel('Probability')
plt.title('CDF of Gaussian Continuous Random Variable')
plt.show()

# Calculate the expectation of the samples
expectation = np.mean(samples)
print(f'Expectation: {expectation}')

# Calculate the variance of the samples
variance = np.var(samples)
print(f'Variance: {variance}')
```

***All of our gratitude for your attentive listening!***